

## 05 — RAG Pipeline (Deep Dive)

**Module purpose:** grounds LLM answers in your own documents (stored via sheet 04) instead of the model's training data. Five files: prompt design, retrieval, single-turn pipeline, conversation memory, and the full multi-turn agent.

### File 1 — rag\_prompts.py (three prompt variants)

```
RAG_PROMPT = ChatPromptTemplate.from_template("""
You are a helpful assistant. Answer the user's question using ONLY the context provided
↳ below.
Rules:
- If the answer is clearly present in the context, answer directly and concisely.
- If the context does not contain enough information, say: "I don't have enough information
↳ to answer this."
- Do NOT make up facts. Do NOT use knowledge outside the provided context.
Context: {context}
User Question: {user_query}
Answer:
""")

CONVERSATIONAL_RAG_PROMPT = ChatPromptTemplate.from_template("""
... Use chat history only to resolve references like "it", "that", or "the above".
Chat History: {chat_history}
Context: {context}
User Question: {user_query}
""")

# A third variant, STRUCTURED_RAG_PROMPT, asks for bullet-point formatted answers – same
↳ grounding rules.
```

**Working explanation:** the single most important line in every RAG prompt used here is the explicit **“say you don’t know”** instruction — this is the primary lever against hallucination. CONVERSATIONAL\_RAG\_PROMPT adds a {chat\_history} slot with a narrow, explicit job: resolve pronouns/references (“it”, “that”), NOT act as an additional source of facts — the grounding rule (“ONLY the context”) still applies to the actual answer content.

### File 2 — context\_retriever.py

```
def retrieve_context(conn, user_query, embedding_model, table_name="VECTOR_STORE",
                    top_k=5, source_filter=None, min_score=0.0) -> list[dict]:
    chunks = similarity_search(conn, user_query, embedding_model, table_name, top_k,
    ↳ source_filter)
    if min_score > 0.0:
        chunks = [c for c in chunks if c["score"] >= min_score]    # drop weakly-related noise
    return chunks

def format_context(chunks: list[dict]) -> str:
    if not chunks:
        return "No relevant context found."
    parts = [f"[{c['rank']}] Source: {c['source_name']} (chunk
    ↳ {c['chunk_index']})\n{c['content']}" for c in chunks]
    return "\n\n".join(parts)    # numbered blocks -> both LLM citation AND UI
    ↳ source display
```

**Working explanation:** two separate, single-purpose functions — `retrieve_context` talks to HANA (sheet 04's `similarity_search`) and applies a **score floor**; `format_context` is a pure string-formatting function with no I/O. Splitting them means you can unit-test formatting without a live DB connection, and swap the retrieval backend without touching formatting logic.

### File 3 — `rag_pipeline.py` (single-turn, no memory)

```
def rag_query(conn, user_query, embedding_model, llm, table_name="VECTOR_STORE",
              top_k=5, source_filter=None, min_score=0.0, prompt=None) -> dict:
    chunks = retrieve_context(conn, user_query, embedding_model, table_name, top_k,
    ↪ source_filter, min_score)
    context = format_context(chunks)
    active_prompt = prompt or RAG_PROMPT
    chain = active_prompt | llm | StrOutputParser()
    answer = chain.invoke({"context": context, "user_query": user_query})
    sources = [{"rank": c["rank"], "source_name": c["source_name"], "score": c["score"]} for
    ↪ c in chunks]
    return {"answer": answer.strip(), "context": context, "sources": sources, "chunks":
    ↪ chunks}
```

**Working explanation:** this function is the textbook 4-step RAG shape — **retrieve, format, generate, cite** — and returns not just the answer but the context actually used and structured sources, so a UI can show “answered from these 3 chunks” without re-deriving anything.

### File 4 — `conversation_memory.py` (ChatMemory class)

```
from langchain.memory import ConversationBufferWindowMemory

class ChatMemory:
    def __init__(self, window=5):
        self._memory = ConversationBufferWindowMemory(k=window, return_messages=True) # k =
        ↪ sliding window size

    def save(self, user_input, ai_output):
        self._memory.save_context({"input": user_input}, {"output": ai_output})

    def get_history_string(self) -> str:
        messages = self._memory.load_memory_variables({})["history"]
        lines = []
        for msg in messages:
            if isinstance(msg, HumanMessage): lines.append(f"User: {msg.content}")
            elif isinstance(msg, AIMessage): lines.append(f"Assistant: {msg.content}")
        return "\n".join(lines) if lines else "No previous conversation."

    def clear(self): self._memory.clear()
    def turn_count(self) -> int: return
    ↪ len(self._memory.load_memory_variables({})["history"]) // 2
```

**Working explanation:** `ConversationBufferWindowMemory(k=5)` automatically keeps only the last **k turns** (each turn = 1 user + 1 assistant message pair) and silently drops older ones — this bounds the token cost of every subsequent call without any manual list-slicing code. This memory object is entirely **client-side, in-process** — it has nothing to do with HANA storage; it's conceptually identical to `ConversationBuffer` in the Orchestration Service sheets (07-08), just implemented

with a LangChain built-in instead of a hand-rolled list.

## File 5 — rag\_agent.py (wires everything: RAGAgent class)

```
class RAGAgent:
    def __init__(self, conn, table_name=None, llm=None, embedding_model=None, top_k=5,
        ↪ min_score=0.0, memory_window=5):
        self.conn, self.table_name = conn, table_name or os.getenv("VECTOR_TABLE_NAME",
            ↪ "VECTOR_STORE")
        self.llm = llm or get_llm()
        self.embedding_model = embedding_model or get_embedding_model()
        self.top_k, self.min_score = top_k, min_score
        self.memory = ChatMemory(window=memory_window)

    def ask(self, user_query, source_filter=None, top_k=None) -> dict:      # one-shot, no
        ↪ memory
        return rag_query(self.conn, user_query, self.embedding_model, self.llm,
            self.table_name, top_k or self.top_k, source_filter, self.min_score)

    def chat(self, user_query, source_filter=None, top_k=None) -> dict:      # conversational,
        ↪ uses memory
        chat_history = self.memory.get_history_string()
        chunks = retrieve_context(self.conn, user_query, self.embedding_model,
        ↪ self.table_name,
            top_k or self.top_k, source_filter, self.min_score)
        context = format_context(chunks)
        chain = CONVERSATIONAL_RAG_PROMPT | self.llm | StrOutputParser()
        answer = chain.invoke({"chat_history": chat_history, "context": context,
        ↪ "user_query": user_query}).strip()
        self.memory.save(user_query, answer)                                # persist this turn BEFORE
        ↪ returning
        sources = [{"rank": c["rank"], "source_name": c["source_name"], "score": c["score"]}
        ↪ for c in chunks]
        return {"answer": answer, "sources": sources, "chat_history": chat_history}

    def clear_memory(self): self.memory.clear()
```

**Working explanation:** .ask() and .chat() are deliberately two different methods, not one method with a flag — .ask() reuses rag\_pipeline.rag\_query (sheet file 3) unchanged for one-shot Q&A; .chat() retrieves context fresh every turn (so follow-ups can pull *new* relevant chunks, not just reuse old ones) but layers in chat\_history so pronouns like “that” resolve correctly.

## Key Points

- The **“say you don’t know”** instruction in the prompt is the single biggest lever against RAG hallucination.
- retrieve\_context/format\_context split (I/O vs. pure formatting) makes the pipeline independently testable.
- min\_score filters weak matches; too high a threshold risks the “no relevant context” fallback firing too often.
- Conversation memory is a **sliding window** — old turns silently drop to bound token growth, the same principle as the Orchestration Service’s ConversationBuffer (sheet 08).
- .chat() re-retrieves context every turn (not cached from the first turn) — critical for follow-up questions that shift topic within the same conversation.

- RAGAgent = rag\_pipeline + context\_retriever + conversation\_memory + rag\_prompts, same “full pipeline in one class” pattern seen in every module.

## Interview Q&A

**Q: What’s the #1 technique shown here to reduce RAG hallucination?** A: An explicit system-prompt instruction to say “I don’t have enough information” when the retrieved context doesn’t answer the question, rather than letting the model fall back on parametric/training knowledge.

**Q: Why does .chat() retrieve context fresh on every turn instead of reusing the first turn’s retrieval?** A: A conversation naturally shifts topics turn to turn — reusing stale context would answer follow-ups using irrelevant chunks; re-retrieving per turn keeps answers grounded in what’s actually relevant to the CURRENT question, while chat\_history handles reference resolution separately.

**Q: How is chat\_history different from the retrieved context, and why keep them separate prompt variables?** A: chat\_history is prior conversation turns (used only to resolve pronouns/references); context is retrieved document chunks (the actual source of facts). Conflating them risks the model treating earlier conversational claims as ground truth instead of the real documents.

**Q: How would you evaluate whether a RAG answer is properly “grounded”?** A: Check whether every factual claim in answer traces back to something in context/sources — manually, or via an automated “faithfulness” check where a second LLM call verifies each answer sentence against the provided context.